

RELATÓRIO DE PROJETO: IMPLEMENTAÇÃO DE GRAFOS EM C#

REDE DE CONTATOS



INTRODUÇÃO

Para representar um grafo digitalmente, duas abordagens são predominantemente utilizadas: a Matriz de Adjacência e a Lista de Adjacência. Enquanto a matriz é ideal para grafos densos (com muitas conexões), a lista de adjacência mostra-se altamente eficiente em termos de consumo de memória para grafos esparsos, que é o cenário típico de redes de contatos, onde cada indivíduo se conecta a apenas uma fração do total de usuários do sistema.

OBJETIVO

O presente projeto tem como objetivo principal aplicar os conceitos teóricos da Teoria dos Grafos no desenvolvimento de um sistema prático e interativo utilizando a linguagem de programação C# e a plataforma .NET.

Especificamente, o trabalho visa:

- Implementar um grafo do tipo não-direcionado (onde as relações de amizade são recíprocas e de via dupla) e não-ponderado (onde todas as conexões possuem o mesmo peso/relevância).
- Utilizar a estrutura de Lista de Adjacência através de coleções nativas da linguagem C# (`Dictionary` e `List`) para otimizar o armazenamento e a busca de dados.
- Desenvolver uma interface de linha de comando (CLI) interativa que permita ao usuário manipular a estrutura em tempo de execução, simulando o dinamismo de uma rede social real através do cadastro de usuários, criação de vínculos e verificação de conexões.



DESENVOLVIMENTO

Modelagem do Problema e Arquitetura do Grafo

O sistema foi projetado mapeando os elementos do problema real diretamente para os conceitos matemáticos de grafos:

- **Vértices:** Representados por strings contendo os nomes únicos das pessoas cadastradas na rede.
- **Arestas:** Representadas pela associação mútua entre duas strings dentro da estrutura de adjacência.

A escolha de um grafo não-direcionado reflete a lógica de uma rede de amizades clássica: se a "Pessoa A" é amiga da "Pessoa B", a reciprocidade é automática. Portanto, ao criar uma aresta, o sistema atualiza a lista de ambas as entidades simultaneamente.

Estrutura de Dados em C#

A eficiência de memória foi alcançada através da declaração da seguinte estrutura no núcleo da classe `RedeSocial`:

```
private Dictionary<string, List<string>> _adjacencias;
```

O uso do `Dictionary` permite que o acesso a qualquer vértice (pessoa) ocorra em tempo constante, ou seja, de complexidade média $O(1)$. Cada chave do dicionário aponta para uma `List<string>`, que armazena os vizinhos imediatos daquele vértice.

Funcionalidades Principais do Algoritmo

O código foi estruturado em métodos modulares para garantir a manutenibilidade e a legibilidade:

1. `AdicionarPessoa(string nome)` : Verifica se o vértice já existe para evitar duplicidade. Caso não exista, inicializa uma nova lista de amigos vazia para o usuário.
2. `AdicionarAmizade(string pessoa1, string pessoa2)`: Localiza ambos os vértices no dicionário. Sendo encontrados, adiciona `pessoa2` à lista de `pessoa1` e vice-versa, garantindo a bidirecionalidade do grafo. O método impede também a auto-referência (um usuário ser amigo de si mesmo).



3. **VerificarAmizade(string pessoa1, string pessoa2):** Realiza uma busca linear simples na lista de adjacência do primeiro vértice para determinar, em tempo de execução, se existe uma aresta direta conectando os dois usuários.
4. **ExibirRede():** Varre toda a estrutura de dados, imprimindo de forma visual cada vértice e seus respectivos nós adjacentes, simulando a plotagem textual do grafo.

Interface Interativa (CLI)

Para conferir dinamismo ao projeto, a função principal (`Main`) foi estruturada ao redor de um laço de repetição contínuo (`while`) alimentado por uma estrutura de controle `switch-case`. Isso permitiu a criação de um menu numérico limpo, onde o usuário interage via teclado, inserindo dados que modificam o grafo em tempo real, aproximando a aplicação acadêmica de um software comercial utilizável.

CONCLUSÃO

A execução deste projeto permitiu consolidar de forma prática os conhecimentos teóricos sobre estruturas de dados avançadas, especificamente a Teoria dos Grafos. A implementação em C# demonstrou que a linguagem oferece recursos robustos de coleções genéricas que simplificam drasticamente a criação de estruturas complexas como a lista de adjacências.

O desenvolvimento de um menu interativo provou-se um diferencial importante, transformando um algoritmo puramente lógico em uma ferramenta visual e de fácil validação. O sistema comportou-se de maneira estável, tratando exceções básicas (como entradas vazias ou tentativas de conectar usuários inexistentes).

Por fim, o projeto cumpre com sucesso o seu papel didático e deixa as portas abertas para expansões futuras. A estrutura base desenvolvida é perfeitamente escalável para a implementação de algoritmos de busca clássicos, como a Busca em Largura (BFS) para descobrir graus de separação entre pessoas, ou a Busca em Profundidade (DFS) para identificar comunidades isoladas dentro da rede social.



MEMBROS

GUSTAVO AMADIO

CAIO VINICIUS

JOÃO MATHEUS VIEIRA

RUY NATHAN

RYAN GABRIEL

GABRIEL PINHEIRO

JADSON VITOR

